# Technical Report on Experimental Implementation in MATLAB/Simulink

Martin Šulák, Marko Zelizňak, Dárius Kočiš

Course: Fuzzy Systems

Topic: Rodríguez-Abreo et al. (2024) – Fuzzy logic controller for UAV with gains op

May 6, 2026

**Abstract**

This technical report documents the experimental implementation of a fuzzy controller for a UAV in MATLAB/Simulink. The work is based on the paper by Rodríguez-Abreo et al. [1], which deals with a fuzzy logic controller for a UAV whose gains are optimized by a genetic algorithm. In this project, a custom dynamic model of a quadrotor UAV was designed using the Newton-Euler description. Fuzzy controllers were then implemented for the $z$, $\phi$, $\theta$, and $\psi$ axes, as well as outer position loops for the $x$ and $y$ axes. The final system was verified in a combined test with references $x_r = 0.2\,\mathrm{m}$, $y_r = 0.2\,\mathrm{m}$, $z_r = 1\,\mathrm{m}$, and $\psi_r = 0\,\mathrm{rad}$.

## Contents

# 1 Introduction

The objective of the assignment in the Fuzzy Systems course was to analyze a scientific paper from the field of fuzzy systems and create an experiment that verifies the principle or functionality of the described solution. The selected topic belongs to the area of intelligent UAV motion control, combining fuzzy logic and evolutionary optimization.

The paper by Rodríguez-Abreo et al. [1] focuses on the design of a fuzzy controller for a UAV, where the controller gains are optimized using a genetic algorithm. In this work, the same idea was experimentally implemented in MATLAB/Simulink. The emphasis was placed on building a custom UAV dynamic model, designing fuzzy controllers, implementing genetic algorithm optimization, and evaluating simulation results.

Throughout the report, the vertical position is denoted as $z$. The symbol $h$ is not used.

# 2 Problem Formulation

A quadrotor UAV is a nonlinear, coupled, and dynamically unstable system. Its control requires regulating altitude, attitude, and position in space. Classical controllers, such as PID controllers, may work with appropriate tuning, but their tuning can be time-consuming and sensitive to the parameters of the model.

A fuzzy controller allows control rules to be expressed using linguistic terms. A genetic algorithm can then be used to automatically tune the gains that scale the inputs and outputs of the fuzzy controller.

The main experimental question was:

> Can a fuzzy controller with GA-optimized gains stabilize a custom Simulink UAV model and track simple references in the $x$, $y$, $z$ axes and in the yaw angle $\psi$?

# 3 Theoretical Background

## 3.1 UAV as a Controlled System

A quadrotor has six degrees of freedom. Its position is described by:

$$x, \quad y, \quad z$$

and its attitude is described by Euler angles:

$$\phi, \quad \theta, \quad \psi,$$

where $\phi$ is roll, $\theta$ is pitch, and $\psi$ is yaw.

The model uses four control inputs:

$$U_1, \quad U_2, \quad U_3, \quad U_4.$$

Their meaning is:

- $U_1$ – total thrust,

- $U_2$ – roll moment,
- $U_3$ – pitch moment,
- $U_4$ – yaw moment.

### 3.2 Fuzzy Controller

The fuzzy controller uses the error and the change of error:

$$e(t) = r(t) - y(t),$$

$$\dot{e}(t) = \frac{d}{dt}e(t).$$

Before entering the fuzzy inference system, the values are scaled:

$$e_F = k_1 e,$$

$$\dot{e}_F = k_2 \dot{e}.$$

The fuzzy output is then scaled by the output gain:

$$u = K_{out} u_F.$$

### 3.3 Genetic Algorithm

A genetic algorithm is a population-based optimization method inspired by natural evolution. Each individual in the population represents one combination of parameters. In this experiment, an individual contained the gain values of a fuzzy controller. The quality of an individual was evaluated using a fitness function that mainly considered:

- RMSE error,
- final error,
- overshoot,
- oscillations after the transient phase,
- magnitude of the control action.

## 4 Custom UAV Dynamic Model

The UAV dynamic model was created as a custom Simulink design. The model was built modularly, allowing translational dynamics, rotational dynamics, motor-speed conversion, and feedback control loops to be tested separately.

## 4.1 Model Parameters

Table 1: UAV model parameters

| Parameter | Value | Meaning |
|---|---|---|
| $m$ | 0.9928 kg | UAV mass |
| $g$ | 9.81 m/s$^2$ | gravitational acceleration |
| $L$ | 0.24 m | arm length |
| $I_{xx}$ | 0.00963 kg m$^2$ | moment of inertia around the $x$-axis |
| $I_{yy}$ | 0.00963 kg m$^2$ | moment of inertia around the $y$-axis |
| $I_{zz}$ | 0.019 kg m$^2$ | moment of inertia around the $z$-axis |
| $b$ | $3.59 \times 10^{-5}$ | thrust coefficient |
| $d$ | $2.081 \times 10^{-6}$ | drag torque coefficient |
| $J_r$ | 0.04439 | rotor inertia |

The hover motor speed was calculated as:

$$\omega_{hover} = \sqrt{\frac{mg}{4b}} = 260.4283 \text{ rad/s}.$$

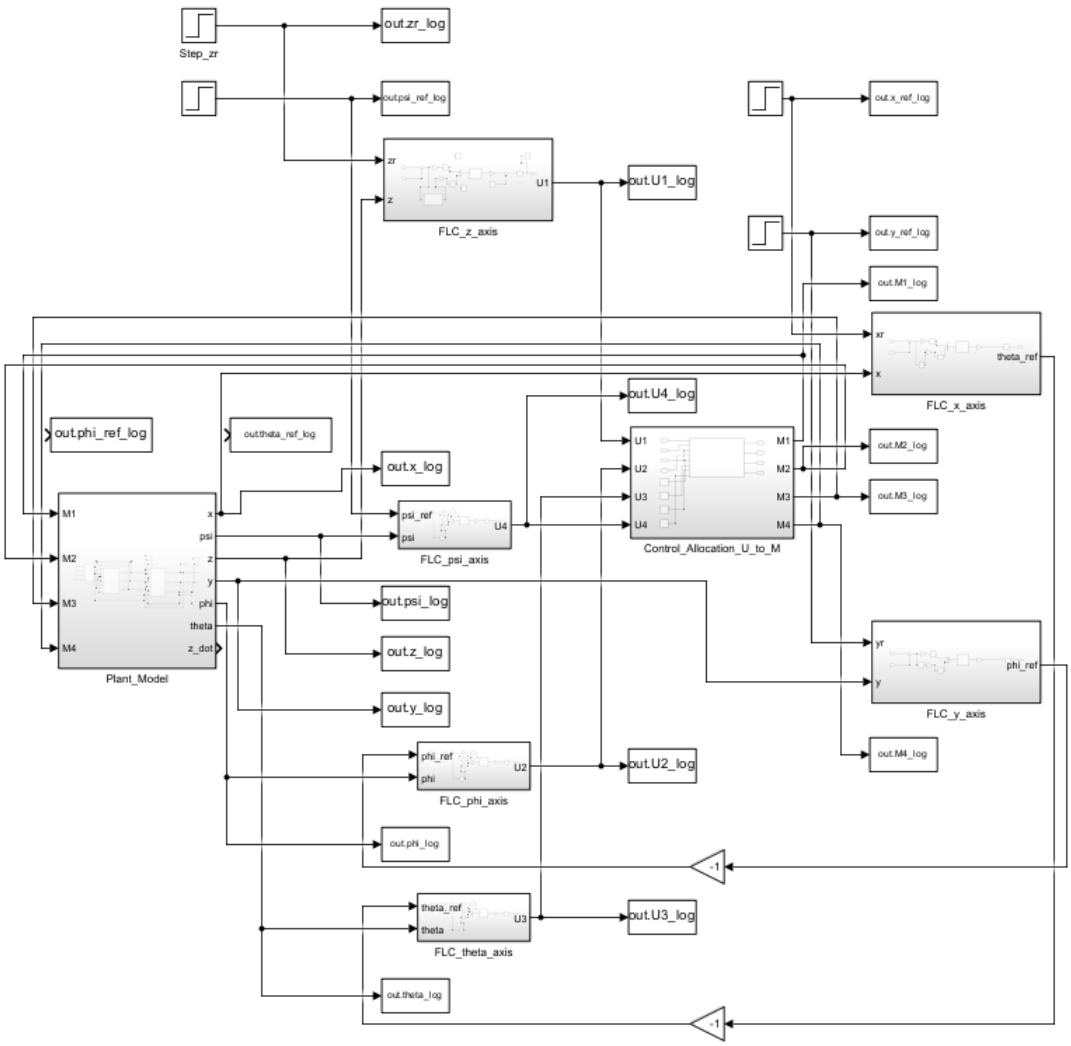


Figure 1: Main Simulink scheme of the custom UAV model with fuzzy controllers.

# 5 Fuzzy Control Structure Design

## 5.1 Structure of One Axis Fuzzy Controller

Each axis controller was implemented according to the same basic structure:

1. calculation of the error between the reference and the actual value,
2. calculation of the derivative of the error,
3. scaling of the error and its derivative,
4. saturation of the inputs to the interval $[-0.95, 0.95]$,
5. signal merging using a Mux block,
6. evaluation of the Mamdani fuzzy system,
7. output scaling using $K_{out}$,

8. saturation of the output control action or reference angle.

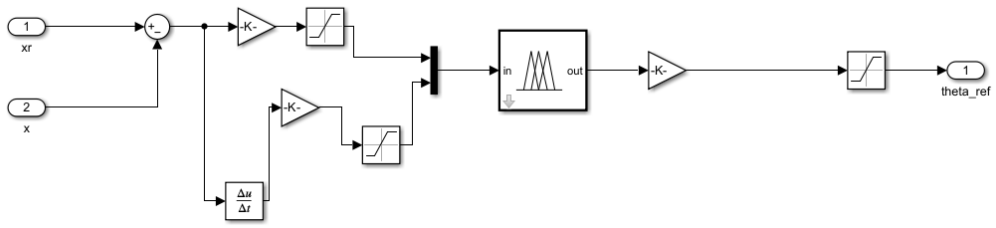


Figure 2: General structure of an axis fuzzy controller.

## 5.2 Fuzzy Inference System

A Mamdani fuzzy system with two inputs and one output was used:

- input 1: error $e$,
- input 2: change of error $\Delta e$,
- output: $u$,
- number of input membership functions: 3 for each input,
- number of output membership functions: 9,
- number of rules: 9,
- defuzzification method: centroid.

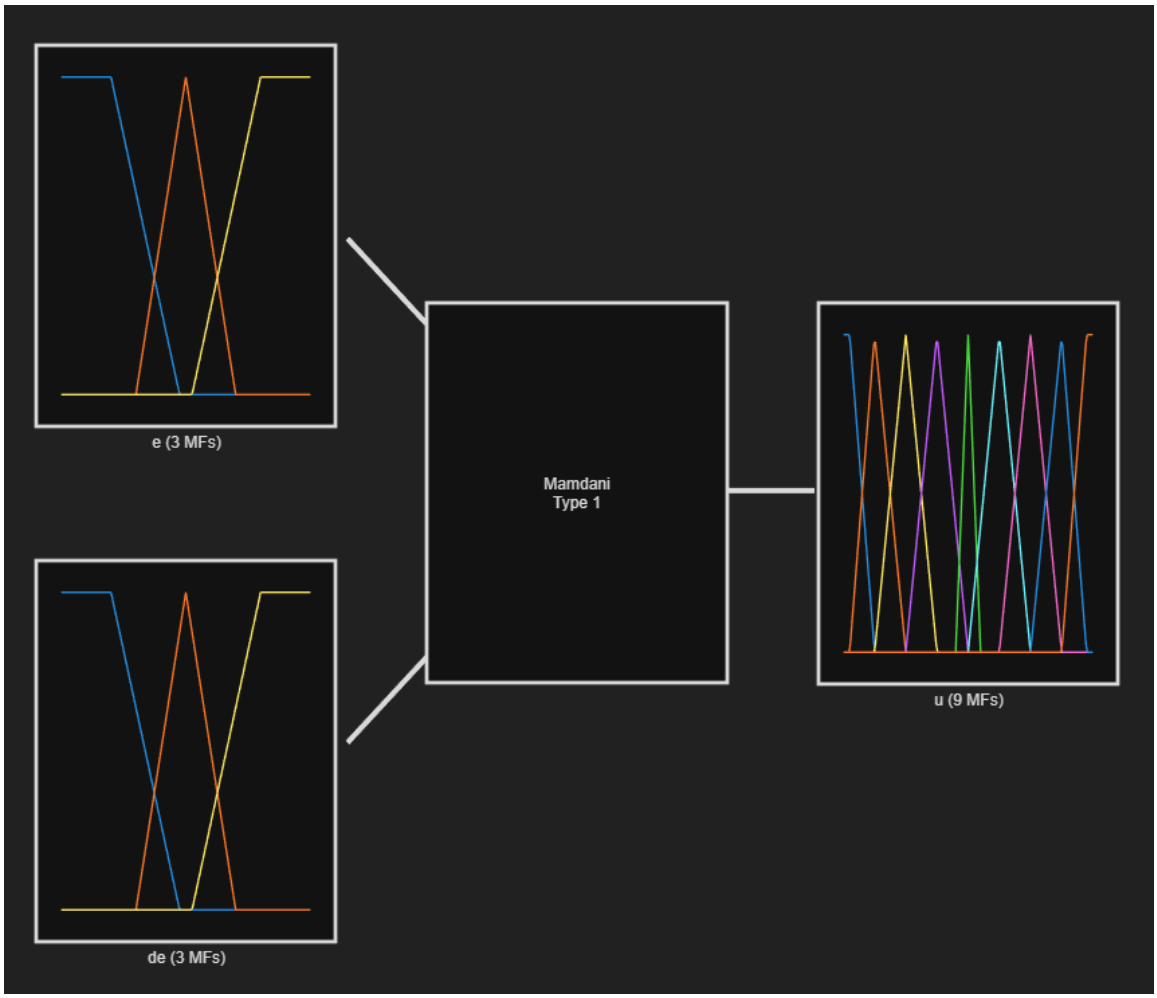


Figure 3: Design of the fuzzy system `fis_axis` in Fuzzy Logic Designer.

## 5.3 Controller Mapping

Table 2: Control loops used in the experiment

| Controller | Inputs | Output | Function |
|---|---|---|---|
| FLC_z | $z_r, z$ | $U_1$ | altitude control |
| FLC_$\phi$ | $\phi_r, \phi$ | $U_2$ | roll control |
| FLC_$\theta$ | $\theta_r, \theta$ | $U_3$ | pitch control |
| FLC_$\psi$ | $\psi_r, \psi$ | $U_4$ | yaw control |
| FLC_x | $x_r, x$ | $\theta_r$ | outer loop for $x$ |
| FLC_y | $y_r, y$ | $\phi_r$ | outer loop for $y$ |

For altitude, the following relation was used:

$$U_1 = mg + K_{out,z} u_{F,z}.$$

For the rotational axes:

$$U_2 = K_{out,\phi} u_{F,\phi},$$
$$U_3 = K_{out,\theta} u_{F,\theta},$$

$$U_4 = K_{out,\psi} u_{F,\psi}.$$

For the outer position loops, sign testing showed that sign inversion was necessary:

$$\theta_r = -K_{out,x} u_{F,x},$$

$$\phi_r = -K_{out,y} u_{F,y}.$$

# 6 Experimental Methodology

The experiment was carried out in several stages:

1. creation of the custom UAV dynamic model,
2. verification of the basic hover behavior,
3. design of the fuzzy controller for the $z$ axis,
4. GA optimization of the $z$ controller,
5. design of controllers for $\phi$, $\theta$, and $\psi$,
6. optimization of the $\phi$ and $\psi$ controllers,
7. use of the same gains for $\theta$ as for $\phi$, because $I_{xx} = I_{yy}$,
8. addition of outer loops for $x$ and $y$,
9. GA optimization of the outer loop using GA_xy,
10. final evaluation of the combined test.

The GA simulations were accelerated using `parsim`. Each individual in the population was evaluated as a separate simulation of the model.

# 7 Fitness Function

The general form of the fitness function was:

$$J = \alpha \cdot RMSE + \beta \cdot e_{final} + \gamma \cdot M_p + \delta \cdot \sigma_{late} + \lambda \cdot \overline{|U|}.$$

In the $x/y$ outer-loop optimization, the fitness function focused mainly on:

- RMSE for $x$ and $y$,
- final error for $x$ and $y$,
- overshoot for $x$ and $y$,
- altitude stability in $z$,
- small errors in $\phi, \theta, \psi$,
- small control actions $U_2$ and $U_3$.

# 8 Final Optimized Parameters

Table 3: Final optimized gains of the fuzzy controllers

| Controller | $k_1$ | $k_2$ | $K_{out}$ |
|---|---|---|---|
| $z$ | 0.350591 | 4.424324 | 3.576077 |
| $\phi$ | 11.549638 | 2.386904 | 0.019897 |
| $\theta$ | 11.549638 | 2.386904 | 0.019897 |
| $\psi$ | 21.238180 | 3.994971 | 0.004575 |
| $x$ | 1.797211 | 3.731435 | 0.047172 |
| $y$ | 2.916350 | 2.307857 | 0.037567 |

The outer-loop limits were:

$$\theta_{ref,max} = 0.034907 \text{ rad},$$

$$\phi_{ref,max} = 0.016155 \text{ rad}.$$

# 9 Experimental Results

## 9.1 Altitude Response

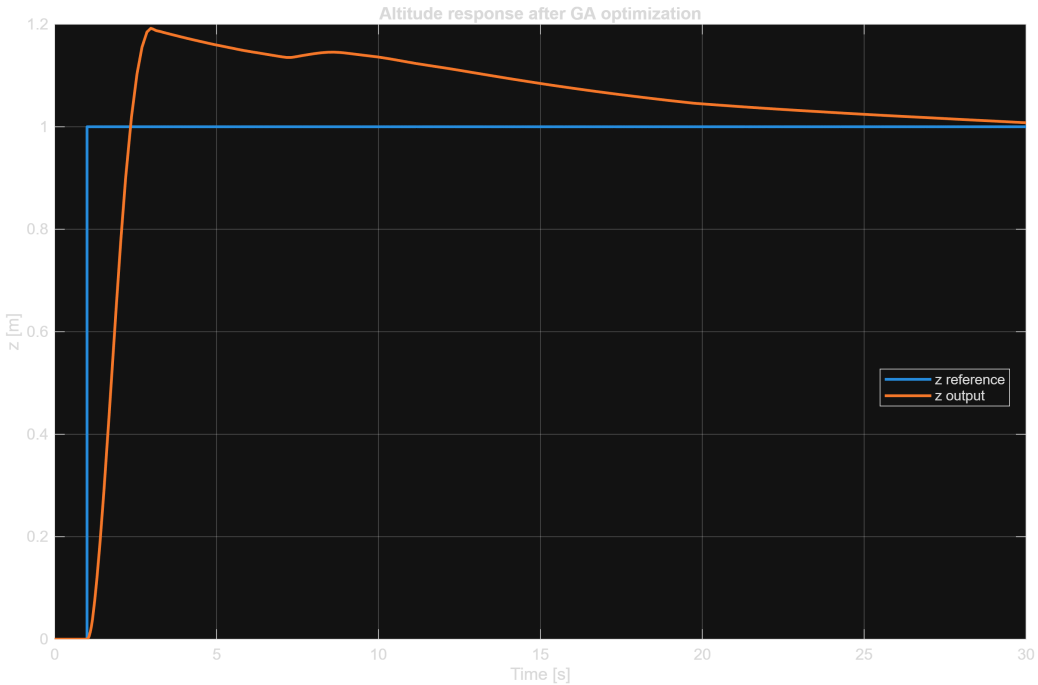


Figure 4: Altitude response $z$ in the final combined test.

## 9.2 Roll Angle During the Final Test


Figure 5: Roll angle response $\phi$ during the final $x - y$ position control test.

## 9.3 Outer-Loop Response for the $x$ Axis

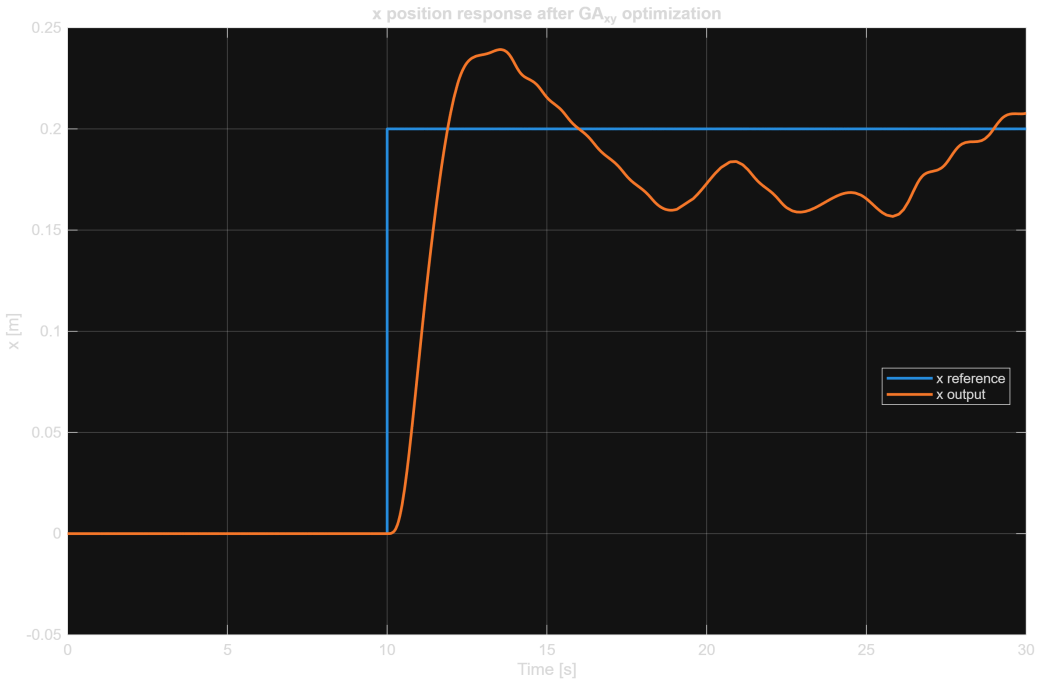


Figure 6: Response of the $x$ axis after GA optimization of the outer loop.

## 9.4 Outer-Loop Response for the $y$ Axis

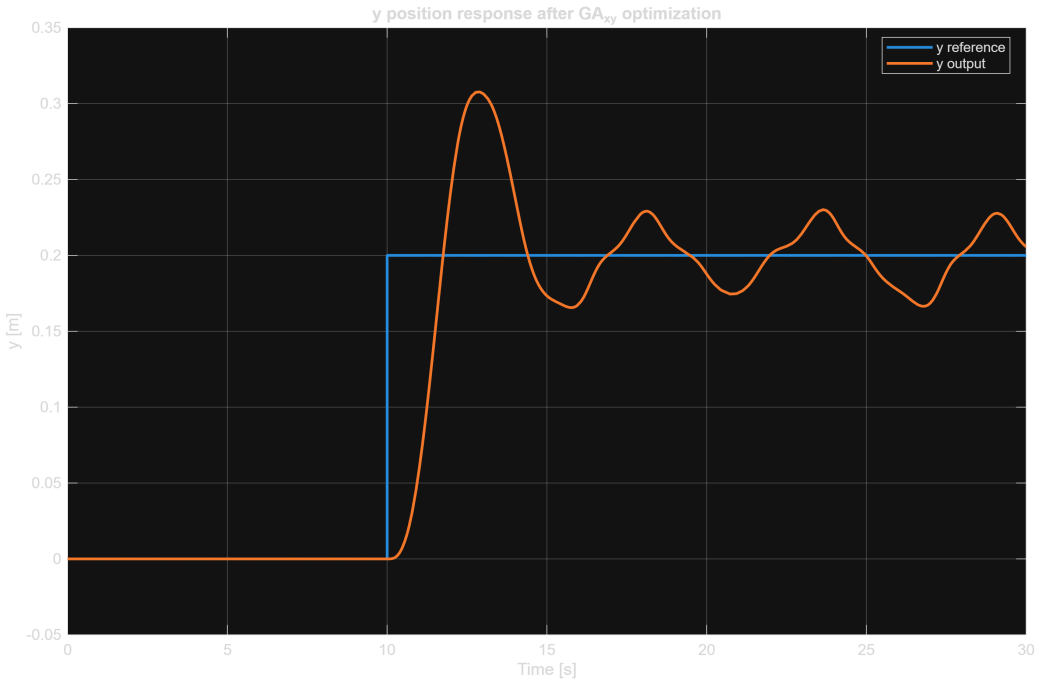


Figure 7: Response of the $y$ axis after GA optimization of the outer loop.

## 9.5 Combined Position Response

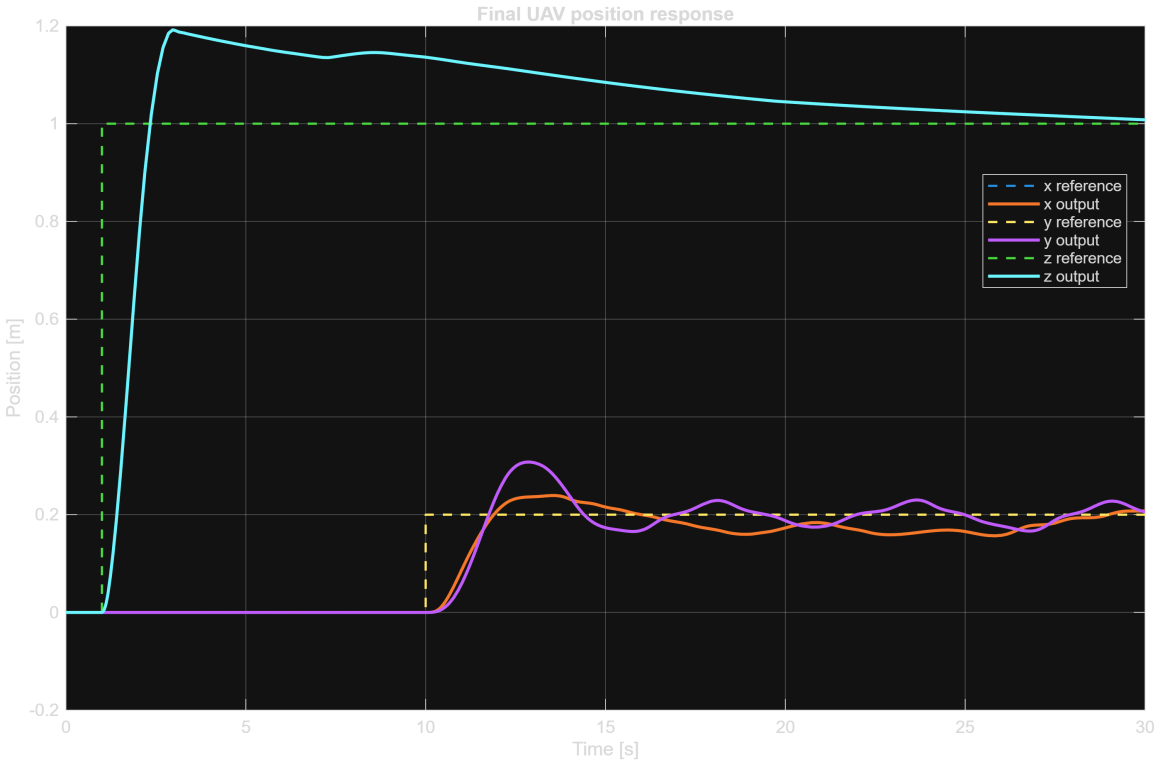


Figure 8: Final combined UAV position response in the $x$, $y$, and $z$ axes.

# 10 Final Combined Test

The final test was configured as follows:

$$x_r = 0.2 \text{ m},$$

$$y_r = 0.2 \text{ m},$$

$$z_r = 1.0 \text{ m},$$

$$\psi_r = 0 \text{ rad}.$$

The step in the $x$ and $y$ axes was applied at 10 s. The step in the $z$ axis was applied at 1 s.

Table 4: Final metrics of the combined test

| Axis | Reference | Final value | Final error | Maximum | RMSE |
|---|---|---|---|---|---|
| $x$ [m] | 0.200000 | 0.207717 | 0.007717 | 0.239218 | 0.054177 |
| $y$ [m] | 0.200000 | 0.205604 | 0.005604 | 0.307690 | 0.060069 |
| $z$ [m] | 1.000000 | 1.007889 | 0.007889 | 1.192164 | 0.172168 |
| $\phi$ [rad] | 0.000000 | -0.003350 | 0.003350 | 0.022947 | 0.008615 |
| $\theta$ [rad] | 0.000000 | -0.003494 | 0.003494 | 0.011924 | 0.006182 |
| $\psi$ [rad] | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 |

## 10.1 Motor Signals

Table 5: Motor values in the final test

| Motor | Final value [rad/s] | Mean value [rad/s] |
|---|---|---|
| $M_1$ | 260.428295 | 260.922917 |
| $M_2$ | 260.428295 | 260.928969 |
| $M_3$ | 260.428295 | 260.895239 |
| $M_4$ | 260.428295 | 260.889064 |

At the end of the simulation, the motor speed difference was zero, meaning that after the transient phase the UAV returned to the hover regime.

# 11 Critical Analysis and Discussion

The experiment showed that a fuzzy controller with GA-optimized gains can stabilize the custom UAV model and track simple references in space. The most significant improvement was achieved after optimizing the $x/y$ outer loop, where the final errors were reduced to the millimeter-to-centimeter range.

The final results were:

$$x = 0.207717 \text{ m},$$

$$y = 0.205604 \text{ m},$$

$$z = 1.007889 \text{ m}.$$

For the references $x_r = 0.2$ m, $y_r = 0.2$ m, and $z_r = 1$ m, these are very small steady-state errors.

The main remaining limitation is the transient response. The $z$ axis had an overshoot of approximately 0.192 m and the $y$ axis had an overshoot of approximately 0.108 m. Nevertheless, the system remained stable and approached the reference values after the transient phase.

# 12 Limitations of the Solution

The solution has several limitations:

- the model is simulation-based and was not validated on a real UAV,
- sensor noise was not included,
- real motor and controller delays were not modeled,
- mainly simple step references were tested,
- GA optimization was performed for selected scenarios and may not be globally optimal.

# 13 Future Work

Future work could include:

- testing the tracking of a more complex trajectory,
- adding measurement noise and disturbance inputs,
- comparing the results with a PID controller,
- optimizing the shape of the membership functions,
- using a multi-scenario fitness function,
- verifying the model in Hardware-in-the-Loop mode or on a real UAV.

# 14 Contribution of Team Members

Table 6: Contribution of team members

| Team member | Contribution |
|---|---|
| Martin Šulák | Design and implementation of the Simulink model, design of fuzzy controllers, GA optimization, diagnostics of results, preparation of the technical report. |
| Marko Zelizňak | Consultation of the solution structure, review of results, and help with preparation of outputs. |
| Dárius Kočiš | Consultation of the experiment, review of processing, and help with documentation. |

The stated distribution can be adjusted according to the real distribution of work in the team.

# 15 Conclusion

A complete fuzzy control system for a UAV was created in MATLAB/Simulink. First, a custom UAV dynamic model was designed. Then, fuzzy controllers were implemented for $z$, $\phi$, $\theta$, $\psi$, $x$, and $y$. A genetic algorithm was used to tune the controller gains.

The final test showed that the UAV model can track the references:

$$x_r = 0.2\,\mathrm{m}, \quad y_r = 0.2\,\mathrm{m}, \quad z_r = 1\,\mathrm{m}$$

with the achieved final values:

$$x = 0.207717\,\mathrm{m},$$

$$y = 0.205604\,\mathrm{m},$$

$$z = 1.007889\,\mathrm{m}.$$

The results confirm that a fuzzy controller with GA-optimized gains can stabilize the custom UAV model and track a simple reference position.

ChatGPT was used as an auxiliary consultation tool during the design of the control structure, creation of MATLAB scripts, debugging of implementation errors, and formulation of the report text. The final simulation values and parameter decisions are based on the author's own MATLAB/Simulink experiments.

# A Selected MATLAB Code

## A.1 Initialization of Final Gains

Listing 1: Example of final parameters in init_uav_params.m

```
m = 0.9928;
g = 9.81;
L = 0.24;

```

```
Ixx = 0.00963;
Iyy = 0.00963;
Izz = 0.019;

b = 3.59e-5;
d = 2.081e-6;
Jr = 0.04439;

omega_hover = sqrt((m*g)/(4*b));

k1_z   = 0.350591;
k2_z   = 4.424324;
Kout_z = 3.576077;

k1_phi   = 11.549638;
k2_phi   = 2.386904;
Kout_phi = 0.019897;

k1_theta   = 11.549638;
k2_theta   = 2.386904;
Kout_theta = 0.019897;

k1_psi   = 21.238180;
k2_psi   = 3.994971;
Kout_psi = 0.004575;

k1_x = 1.797211;
k2_x = 3.731435;
Kout_x = 0.047172;
theta_ref_max = 0.034907;

k1_y = 2.916350;
k2_y = 2.307857;
Kout_y = 0.037567;
phi_ref_max = 0.016155;
```

## A.2 Creation of the Fuzzy System

Listing 2: Shortened principle of FIS creation

```
fis_axis = mamfis("Name","fis_uav_axis");

fis_axis = addInput(fis_axis,[-1 1],"Name","e");
fis_axis = addMF(fis_axis,"e","trapmf",[-1 -1 -0.55 -0.10],"Name","N");
fis_axis = addMF(fis_axis,"e","trimf",[-0.35 0 0.35],"Name","C");
fis_axis = addMF(fis_axis,"e","trapmf",[0.10 0.55 1 1],"Name","P");

fis_axis = addInput(fis_axis,[-1 1],"Name","de");
% same membership functions for de

fis_axis = addOutput(fis_axis,[-1 1],"Name","u");
% output MFs: NB4, NB3, NB2, NB1, Z, PB1, PB2, PB3, PB4

ruleList = [
    1 1 1 1 1
    1 2 2 1 1
    1 3 3 1 1
```

```
    2 1 4 1 1
    2 2 5 1 1
    2 3 6 1 1
    3 1 7 1 1
    3 2 8 1 1
    3 3 9 1 1
];

fis_axis = addRule(fis_axis,ruleList);
writeFIS(fis_axis,"fis_uav_axis.fis");
```

## A.3 Using parsim in GA

Listing 3: Principle of parallel population evaluation

```
for i = 1:popSize
    in(i) = Simulink.SimulationInput(model);

    in(i) = in(i).setModelParameter( ...
        "StopTime", stopTime, ...
        "SimulationMode", "accelerator", ...
        "ReturnWorkspaceOutputs", "on");

    in(i) = in(i).setVariable("k1_x", population(i,1));
    in(i) = in(i).setVariable("k2_x", population(i,2));
    in(i) = in(i).setVariable("Kout_x", population(i,3));
    in(i) = in(i).setVariable("theta_ref_max", population(i,4));

    in(i) = in(i).setVariable("k1_y", population(i,5));
    in(i) = in(i).setVariable("k2_y", population(i,6));
    in(i) = in(i).setVariable("Kout_y", population(i,7));
    in(i) = in(i).setVariable("phi_ref_max", population(i,8));
end

simOut = parsim(in, ...
    "ShowProgress", "on", ...
    "ShowSimulationManager", "off", ...
    "UseFastRestart", "on", ...
    "StopOnError", "off");
```

## A.4 Fitness Function for GA_xy

Listing 4: Fitness function for x/y outer-loop optimization

```
fitness(i) = ...
    10.0 * metrics.xRmse + ...
    20.0 * metrics.xFinalError + ...
    12.0 * metrics.xOvershoot + ...
    5.0  * metrics.xLateStd + ...
    10.0 * metrics.yRmse + ...
    20.0 * metrics.yFinalError + ...
    12.0 * metrics.yOvershoot + ...
    5.0  * metrics.yLateStd + ...
    4.0  * metrics.zFinalError + ...
    2.0  * metrics.zRmse + ...
```

```
    2.0  * metrics.phiFinalError + ...
    2.0  * metrics.thetaFinalError + ...
    2.0  * metrics.psiFinalError + ...
    0.5  * metrics.meanAbsU2 + ...
    0.5  * metrics.meanAbsU3;
```

## A.5 Shortened Final Diagnostic Output

Listing 5: Final diagnostic output

```
Axis x:
  ref_final    = 0.200000
  output_final = 0.207717
  final_error  = 0.007717
  max_output   = 0.239218

Axis y:
  ref_final    = 0.200000
  output_final = 0.205604
  final_error  = 0.005604
  max_output   = 0.307690

Axis z:
  ref_final    = 1.000000
  output_final = 1.007889
  final_error  = 0.007889
  max_output   = 1.192164

Motor final:
  M1 = M2 = M3 = M4 = 260.428295 rad/s
```

# References


[1] O. Rodríguez-Abreo, J. Rodríguez-Reséndiz, A. García-Cerezo, J. R. García-Martínez: *Fuzzy logic controller for UAV with gains optimized via genetic algorithm*. Heliyon, 10, e26363, 2024. DOI: https://doi.org/10.1016/j.heliyon.2024.e26363

[2] Assignment instructions for the Fuzzy Systems course – 2026. Internal course document.